Document number: P0077R0
Supersedes: N4446
Date: 2015-09-22
Project: Programming Language C++, Library Evolution Working Group
Reply-to: Agustín Bergé agustinberge@gmail.com

`is_callable`, the missing *INVOKE* related trait

0. History

- Add discussion on alternative syntax.
- Add discussion on additional nothrow trait, add `is_nothrow_callable`.
- Add feature-testing macro recomendation.
- Remove discussion on naming.

1. Introduction

This paper proposes to introduce a new trait to determine whether an *INVOKE* expression is well formed.

2. Motivation

Starting with C++11, the library introduced the pseudo-macro *INVOKE* as a way to uniformly handle function objects and member pointers as call expressions. The trait `result_of` was made to follow *INVOKE* semantics as well. This left users —who want to follow the precedence set forth by the standard library— with the correct result type but no direct way of obtaining such result, and `invoke` implementations proliferated.

This was recently rectified by the introduction of `invoke` to the working draft [N4169]. However, there is still one piece of the puzzle missing, and is the ability to query whether an *INVOKE* expression is well formed when treated as an unevaluated operand. Such functionality is currently present in the form of C++14 SFINAE-friendly `result_of`, albeit in a non user-friendly way, and it should be made readily available in trait form for the same reasons `invoke` was introduced into the library.

The following is an artist depiction of such trait:

```
template <class T, class R = void, class = void>
struct is_callable
  : false_type
{};

template <class T>
struct is_callable<T, void, void_t<result_of_t<T>>>
  : true_type
{};

template <class T, class R>
struct is_callable<T, R, void_t<result_of_t<T>>>
  : is_convertible<result_of_t<T>, R>
{};
```

This trait is implemented in the wild under different names, and the check for a compatible result type is not always present. This post [call-me-maybe] shows how the implementation of such trait has been both improved and simplified by every new standard.

3. Design questions

3.1 Compatible return types

INVOKE comes in two flavors, the primary `INVOKE(f, t1, t2, ..., tN)` and `INVOKE(f, t1, t2, ..., tN, R)` defined as `INVOKE(f, t1, t2, ..., tN)` implicitly converted to R. After the resolution of LWG2420, `INVOKE(f, t1, t2, ..., tN, void)` is defined as `static_cast<void>(INVOKE(f, t1, t2, ..., tN))`. Both flavors can be supported with a defaulted template argument:

```
template <class, class R = void> struct is_callable; // not defined
template <class Fn, class... ArgTypes, class R>
  struct is_callable<Fn(ArgTypes...), R>;
```

However, if only one of those flavors would be supported there would be no missing functionality, only more work for the user.

3.2 Alternative parameter syntax

> Someone suggests alternative parameter syntax, `is_callable<Fn, R(Args...)>`.
>
> Do we want AB to add consideration of the alternative syntax in the paper?
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 0 | 6 | 5 | 0 | 0 |
>
> But consistency between this and invocation_traits (in Fundamentals v1) is important.

The syntax used by the invocation traits in the Library Fundamentals TS is `Fn(Args...)`, which is consistent with the syntax used by `std::result_of`. The optional trailing `R` for a checked compatible return type is consistent with the alternative flavor of *`INVOKE`*.

```
template <class Fn, class... ArgTypes>
struct invocation_type<Fn(ArgTypes...)>;

template <class Fn, class... ArgTypes>
struct result_of<Fn(ArgTypes...)>;
```

For consistency with the rest of the standard library, the suggested syntax is `Fn(ArgTypes...)` for representing an instance of a callable `Fn` invoked with arguments of type `ArgTypes...`.

3.3 Additional nothrow trait

> Add `is_noexcept_callable`?
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 1 | 4 | 4 | 2 | 0 |

Traits that check whether certain expressions involving special member functions are well-formed also ship an additional `nothrow` trait, that reports the result of applying the `noexcept` operator to the expression. It is reasonable to provide a similar additional `nothrow` trait for `is_callable`, `is_nothrow_callable`, that reports whether the given *`INVOKE`* expression is known not to throw any exceptions.

It should be noted that the standard library does not specify an exception specification for its callable types (like `reference_wrapper`), but a conforming implementation may add a non-throwing noexcept-specification. The result of `is_nothrow_callable` when a standard library callable type is involved is thus implementation defined.

4. Feature-testing

For the purposes of SG10, we recommend a feature-testing macro named `__cpp_lib_experimental_is_callable`.

5. Proposed Wording

This wording is relative to [N4527].

Change 20.10.2 [meta.type.synop], header `<type_traits>` synopsis, as indicated

```
namespace std {
    [...]
    // 20.10.4.3, type properties:
    [...]
    template <class T> struct is_nothrow_destructible;
    template <class T> struct has_virtual_destructor;

    template <class, class R = void> struct is_callable; // not defined
    template <class Fn, class... ArgTypes, class R>
      struct is_callable<Fn(ArgTypes...), R>;
    template <class, class R = void> struct is_nothrow_callable; // not def
    template <class Fn, class... ArgTypes, class R>
      struct is_nothrow_callable<Fn(ArgTypes...), R>;

    [...]
  }
```

Change 20.10.4.3 [meta.unary.prop], Table 49 — Type property predicates, add new rows with the following content:

Template:

```
template <class Fn, class... ArgTypes, class R>
  struct is_callable<Fn(ArgTypes...), R>;
```

Condition:

> The expression `INVOKE(declval<Fn>(), declval<ArgTypes>()..., R)` is well formed when treated as an unevaluated operand.

Preconditions:

> `Fn` and all types in the parameter pack `ArgTypes` shall be complete types, (possibly cv-qualified) `void`, or arrays of unknown bound.

Template:

```
template <class Fn, class... ArgTypes, class R>
  struct is_nothrow_callable<Fn(ArgTypes...), R>;
```

Condition:

> `is_callable<Fn(ArgTypes...), R>::value` is `true` and the expression `INVOKE(declval<Fn>(), declval<ArgTypes>()..., R)` is known not to throw any exception.

Preconditions:

> `Fn` and all types in the parameter pack `ArgTypes` shall be complete types, (possibly cv-qualified) `void`, or arrays of unknown bound.

6. References

- [N4527] ISO/IEC JTC1 SC22 WG21, Programming Languages - C++, working draft, May 2015
  http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4527.pdf
- [call-me-maybe] True Story: Call Me Maybe - Tales of C++
  http://talesofcpp.fusionfenix.com/post-11/true-story-call-me-maybe
- [N4169] A proposal to add invoke function template (Revision 1) - Tomasz Kaminski
  http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4169.html